K-Means Clustering using Mall Customers Dataset

Practical Exercise

Problem Statement

A shopping mall wants to understand the behaviour of its customers and divide them into meaningful groups. The dataset contains customer information such as Customer ID, Gender, Age, Annual Income and Spending Score.

Unlike classification problems, there is **no target variable**. Our objective is to use **K-Means Clustering** to identify groups of customers with similar characteristics.

For the main clustering exercise, we use **Annual Income** and **Spending Score** because these features allow us to clearly visualize and interpret customer segments.

1. Data Collection and Data Preprocessing

1.1 Importing the Libraries

We first import the required libraries for data handling, visualization, clustering and evaluation.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

1.2 Importing the Dataset

Load the Mall Customers dataset using Pandas. Make sure the CSV file is available in the same folder as this notebook.

In [3]:
```
df = pd.read_csv("Mall_Customers.csv")
df.head()
```

Out[3]:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

1.3 Understanding the Dataset

Each row represents a customer, while each column represents information about that customer's demographic or spending behaviour.

In [4]:
```
print("Shape of the dataset:", df.shape)
print("\nColumn names:")
print(df.columns)
print("\nDataset information:")
df.info()
```

```
Shape of the dataset: (200, 5)

Column names:
Index(['CustomerID', 'Genre', 'Age', 'Annual Income (k$)',
       'Spending Score (1-100)'],
      dtype='str')

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Genre                   200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 8.9 KB
```

Dataset Features

Feature	Description
CustomerID	Unique identification number of the customer
Gender	Gender of the customer
Age	Age of the customer
Annual Income (k$)	Annual income of the customer
Spending Score (1-100)	Score representing customer spending behaviour

Important Observation

In supervised learning, we normally have **X as independent variables and y as the target variable**. In K-Means Clustering, there is **no target variable**. The algorithm tries to discover hidden patterns and groups within the data.

1.4 Checking for Null Values

K-Means cannot directly work with missing values, so we check the dataset before clustering.

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

1.5 Checking for Duplicate Values

Duplicate records may give unnecessary importance to certain observations and may influence the clustering result.

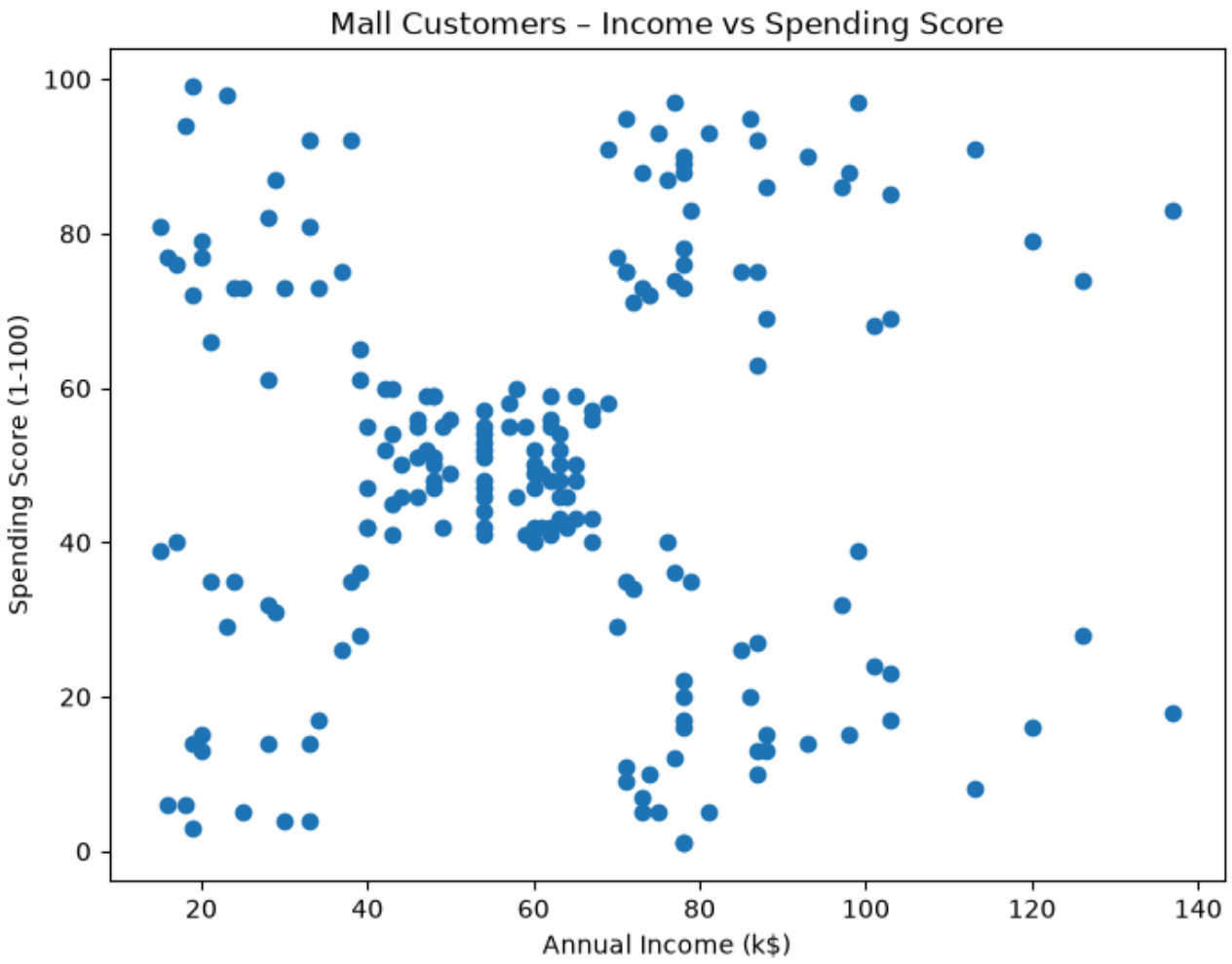
In [6]:
```
df.duplicated().sum()
```

Out[6]:
```
np.int64(0)
```

1.6 Data Visualization

Let us visualize the relationship between **Annual Income** and **Spending Score** before applying K-Means.

In [7]:
```
plt.figure(figsize=(8, 6))
plt.scatter(df['Annual Income (k$)'], df['Spending Score (1-100)'])
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Mall Customers – Income vs Spending Score")
plt.show()
```



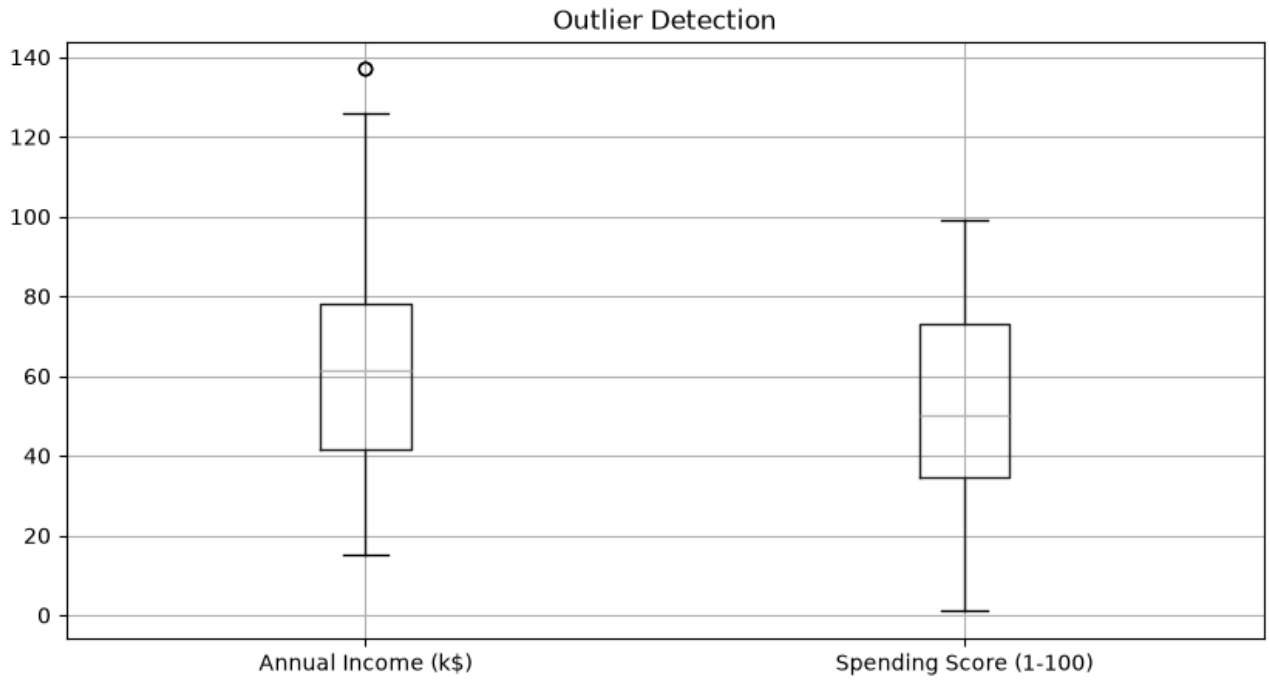
Can you visually identify possible groups of customers from this graph?

Students may observe that customers appear to form different groups. Instead of manually creating those groups, we will use K-Means Clustering.

1.7 Outlier Detection

Since K-Means is distance-based, extreme observations can influence the position of the centroids.

In [8]:
```
plt.figure(figsize=(10, 5))
df[['Annual Income (k$)', 'Spending Score (1-100)']].boxplot()
plt.title("Outlier Detection")
plt.show()
```



Important Note

Do not automatically remove every outlier. An unusual customer may represent a valid and important customer segment. Any removal should be supported by proper domain or business understanding.

1.8 Selecting Features for Clustering

For this exercise, we will use **Annual Income** and **Spending Score** to answer the question: **Can customers be grouped based on their income and spending behaviour?**

In [9]:
```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
X.head()
```

Out[9]:

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

1.9 Feature Scaling

K-Means is a **distance-based algorithm**. If one feature has a much larger numerical range, it can dominate the distance calculation.

We use Standardization:

z = (x − μ) / σ

Feature scaling helps ensure that both selected features contribute appropriately to clustering.

In [10…
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

2. K-Means Clustering

2.1 What is K-Means?

K-Means is an **unsupervised machine learning algorithm** used to divide similar observations into groups called clusters.

The value **K** represents the number of clusters we want to create.

K = 2 means 2 customer groups, K = 3 means 3 customer groups, and K = 5 means 5 customer groups.

Centroids

Each cluster has a central point called a **centroid**. Initially, K-Means selects K centroids, and each customer is assigned to the nearest centroid.

2.3 How Does K-Means Work?

Select the Value of K

↓

Initialize K Centroids

↓

Calculate the Distance of Each Point from the Centroids

↓

Assign Each Point to the Nearest Centroid

↓

Recalculate the Centroids

↓

Repeat the Process

↓

Stop When the Clusters Become Stable

3. Choosing the Optimal Value of K

3.1 Why Do We Need to Choose K?

A very small K may combine different types of customers into the same cluster. A very large K may create too many small and difficult-to-interpret groups.

The **Elbow Method** is commonly used to select a reasonable value of K.

Understanding WCSS

WCSS stands for **Within-Cluster Sum of Squares**. It measures how close data points are to the centroid of their assigned cluster.

As K increases, WCSS generally decreases. However, increasing K indefinitely is not useful.

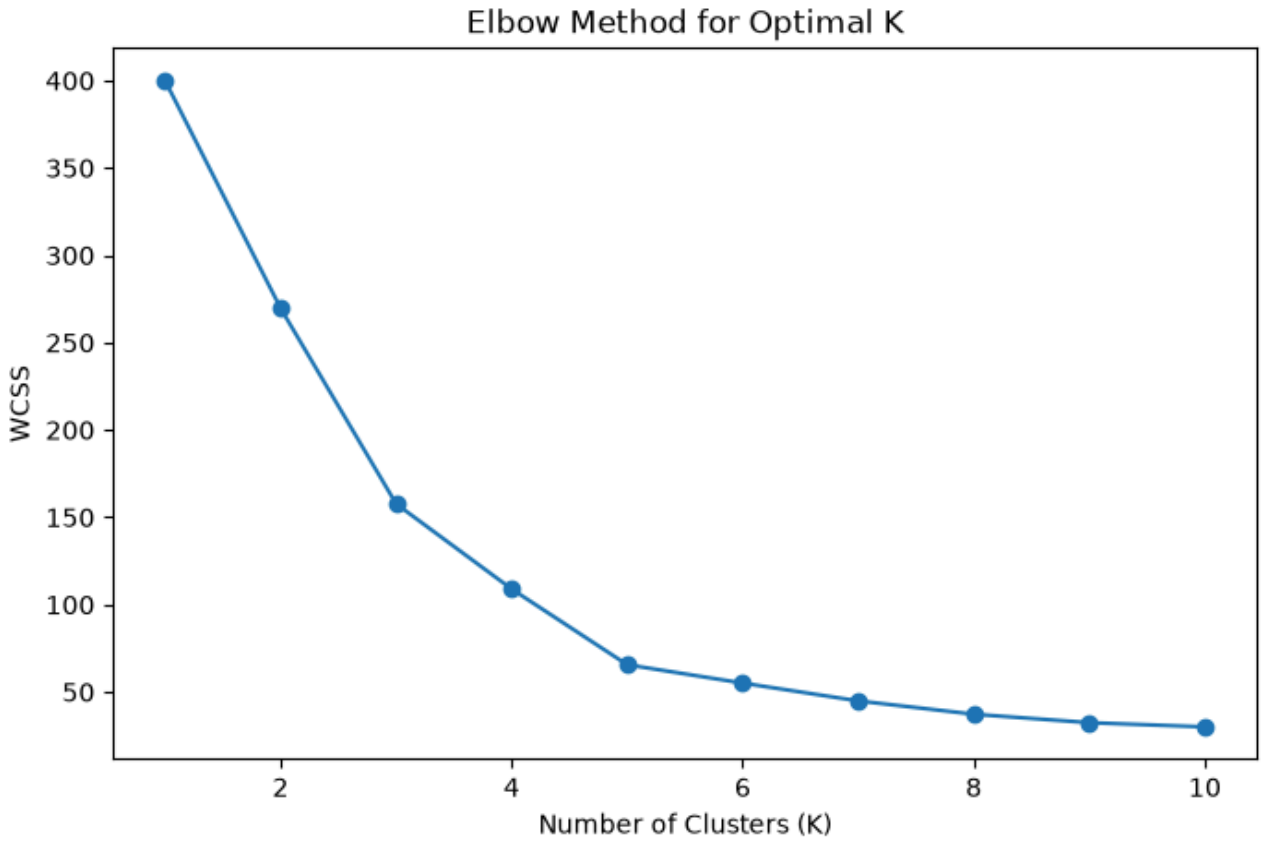
3.3 Applying the Elbow Method

We calculate WCSS for different values of K.

In [11…
```
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```

In [12…
```
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method for Optimal K")
plt.show()
```



3.4 Identifying the Elbow Point

Initially, increasing K significantly reduces WCSS. After a certain point, the improvement becomes smaller. This bend is called the **Elbow Point**.

For the Mall Customers dataset, K is commonly observed around **5** using these two features, but students should select K based on the graph they obtain.

The Elbow Method should be considered together with visualization, Silhouette Score and business understanding.

4. Model Building

4.1 Initializing the K-Means Model

Assume the Elbow Method suggests **K = 5**.

In [13…
```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)
```

Understanding the Parameters

n_clusters = 5 means the algorithm will create five customer groups.

random_state = 42 helps reproduce the same result.

n_init = 10 allows K-Means to try multiple initial centroid positions and select a good solution.

4.2 Training the Model

During training, the algorithm initializes centroids, assigns customers, recalculates centroids and repeats the process until convergence.

In [14…
```
kmeans.fit(X_scaled)
```

Out[14]:

▾ KMeans

Parameters

	n_clusters	5
	n_init	10
	random_state	42
	init	'k-means++'
	max_iter	300
	tol	0.0001
	verbose	0
	copy_x	True
	algorithm	'lloyd'

Fitted attributes

Name	Type	Value
cluster_centers_	ndarray[float64](5, 2)	[[-0.2 ,-0.03], [0.99, 1.24], [-1.33, 1.13], [1.06,-1.28], [-1.31,-1.14]]
inertia_	float	65.57
labels_	ndarray[int32](200,)	[4,2,4,...,1,3,1]
n_features_in_	int	2
n_iter_	int	4

▸ 5 features

kmeans0
kmeans1
kmeans2
kmeans3
kmeans4

4.3 Assigning Cluster Labels

`fit_predict()` trains the model and returns a cluster label for every customer.

In [15…

```
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters
df.head()
```

Out[15]:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

5. Visualizing the Clusters

Cluster Labels

Labels such as 0, 1, 2, 3 and 4 are only identifiers. Cluster 4 is not better than Cluster 1; the numbers do not represent ranking.

Because clustering was performed on scaled data, the centroid coordinates are converted back to the original scale before plotting.

In [16…

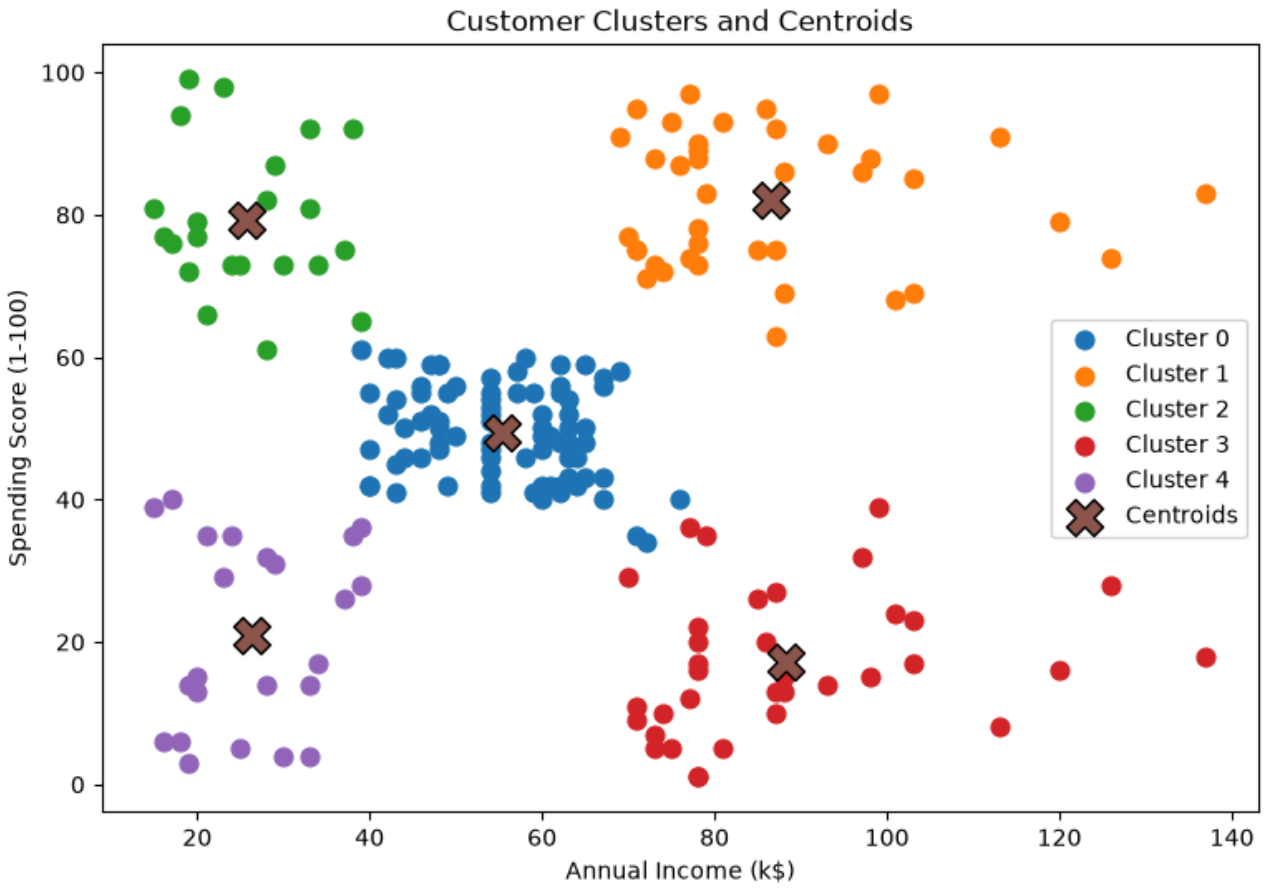
```
centroids_original = scaler.inverse_transform(kmeans.cluster_centers_)

plt.figure(figsize=(9, 6))

for cluster in sorted(df['Cluster'].unique()):
    cluster_data = df[df['Cluster'] == cluster]
    plt.scatter(
        cluster_data['Annual Income (k$)'],
        cluster_data['Spending Score (1-100)'],
        label=f'Cluster {cluster}',
        s=60
    )

plt.scatter(
    centroids_original[:, 0],
    centroids_original[:, 1],
    marker='X',
    s=250,
    edgecolors='black',
    label='Centroids'
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Clusters and Centroids")
plt.legend()
plt.show()
```



#

Which cluster contains high-income and high-spending customers? Which cluster contains high-income but low-spending customers? Which cluster contains low-income and high-spending customers?

6. Cluster Analysis and Interpretation

6.1 Calculating Cluster-Wise Averages

The most important step is understanding what each cluster represents.

In [17…

```
cluster_summary = df.groupby('Cluster')[
    ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
].mean()

cluster_summary
```

Out[17]:

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	42.716049	55.296296	49.518519
1	32.692308	86.538462	82.128205
2	25.272727	25.727273	79.363636
3	41.114286	88.200000	17.114286
4	45.217391	26.304348	20.913043

6.2 Counting the Number of Customers in Each Cluster

In [18…
```
df['Cluster'].value_counts().sort_index()
```

Out[18]:
```
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

6.3 Assigning Meaningful Customer Segments

Based on the average values, clusters may represent:

Customer Segment	Annual Income	Spending Behaviour
Segment A	High	High
Segment B	High	Low
Segment C	Low	High
Segment D	Low	Low
Segment E	Medium	Medium

Always inspect the actual cluster averages before assigning business labels. Do not interpret a cluster based only on its numerical label.

7. Evaluation of Clustering

7.1 Why Accuracy Cannot Be Used

K-Means has no actual target labels, so Accuracy, Precision, Recall and F1-score cannot be directly calculated in the usual classification sense.

7.2 Understanding the Silhouette Score

The Silhouette Score evaluates how well observations fit within their assigned cluster compared with other clusters.

A value closer to 1 generally indicates better separation, a value around 0 suggests overlapping clusters, and negative values can indicate poor assignments.

In [19…
```
silhouette = silhouette_score(X_scaled, clusters)
print("Silhouette Score:", silhouette)
```

```
Silhouette Score: 0.5546571631111091
```

Understanding the Effect of K

Small K vs Large K

Small K	Large K
Fewer clusters	More clusters
Broad customer groups	More specific groups
Easier to interpret	More difficult to interpret
May hide meaningful differences	May create unnecessary fragmentation

8. Exercise

8.1 Experiment with Different Values of K

Build K-Means models using K = 2, 3, 4, 5 and 6. For each model, visualize the clusters, observe the cluster structure, compare WCSS and calculate the Silhouette Score.

Question: Which value of K provides the most meaningful customer segmentation?

8.2 Add Age as an Additional Feature

Use Age, Annual Income and Spending Score. Apply feature scaling and build K-Means again.

Questions: Does adding Age change the cluster structure? Does the Silhouette Score improve? Are the new segments easier to interpret?

8.3 Compare Different Feature Combinations

Try:

Age + Spending Score

Annual Income + Spending Score

Age + Annual Income + Spending Score

Question: Which combination creates the most meaningful customer segments?

Final Takeaways

K-Means is an unsupervised learning algorithm used to group similar observations into clusters.

There is no target variable in clustering.

K represents the number of clusters.

K-Means assigns each data point to the nearest centroid.

Centroids are recalculated repeatedly until the clusters stabilize.

Feature scaling is important because K-Means is distance-based.

The Elbow Method helps identify a reasonable value of K using WCSS.

The Silhouette Score helps evaluate cluster separation and compactness.

A small K may combine different groups together, while a large K may create too many small groups.

The most important final step is not just obtaining cluster numbers but interpreting what each customer cluster actually represents.